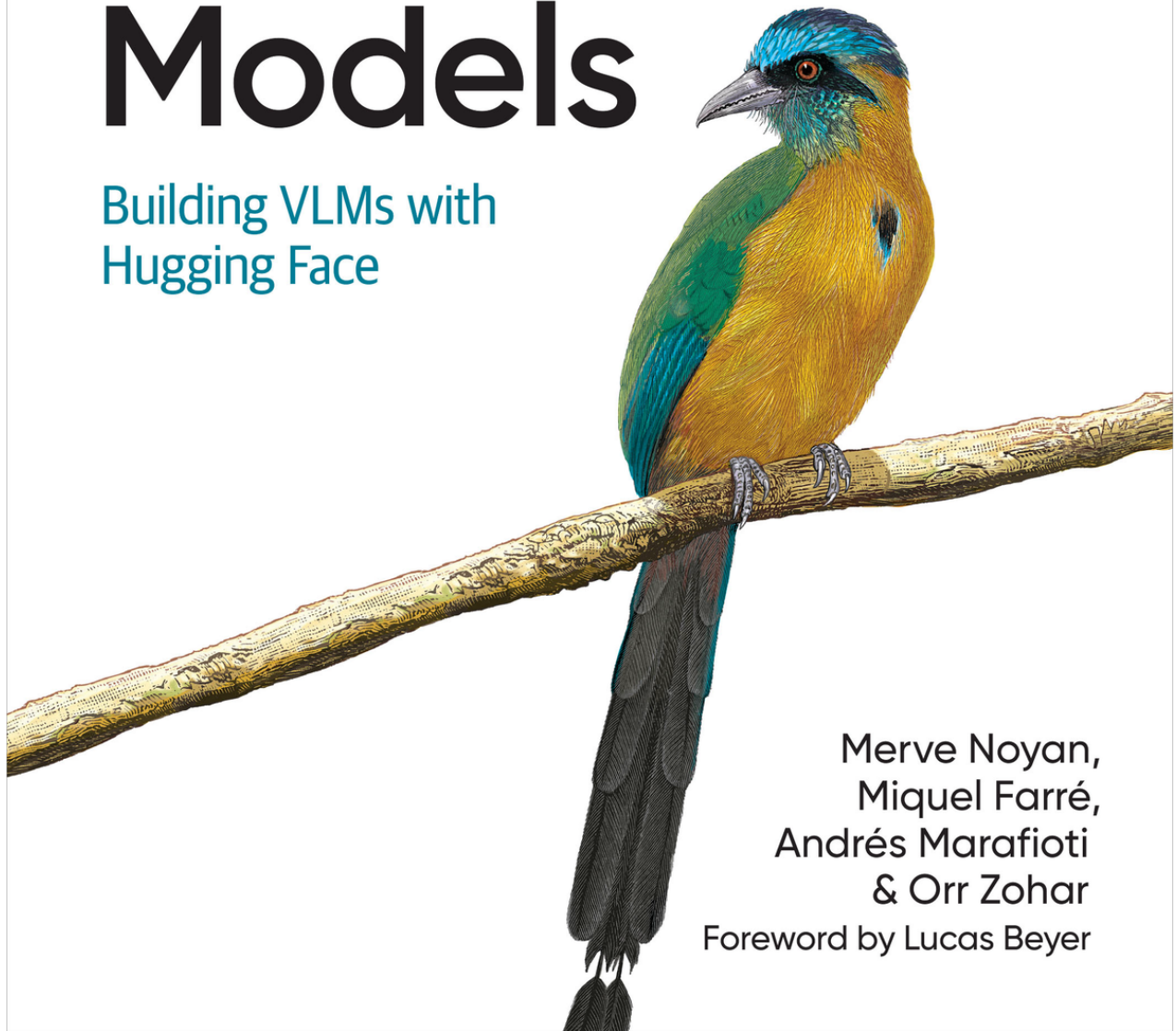


O'REILLY®

Vision Language Models

Building VLMs with
Hugging Face



Merve Noyan,
Miquel Farré,
Andrés Marafioti
& Orr Zohar

Foreword by Lucas Beyer

Praise for *Vision Language Models*

This is an outstanding, comprehensive treatment of the cornerstone frameworks driving modern multimodal AI. The authors have done a remarkable job blending the right amount of foundational theory with deep practical insights while always keeping the user firmly in mind.

—Romann M. Weber, director, research and development, Disney Research Studios

Vision and language used to be separate fields, but thanks to this book, they no longer are. I recommend it to any practitioner who wants to build with vision language models without reading fifty papers first.

—Lewis Tunstall, research scientist,
Hugging Face

A great hands-on resource and practical breakdown of the VLM stack—highly recommended reading for anyone starting to build and use modern vision language models!

—Andreas Steiner, research engineer,
Google DeepMind

This book is the perfect bridge between VLM theory and application. It's an essential, hands-on guide for any developer looking to build and deploy modern, efficient multimodal systems.

—Omar Sanseviero, developer experience lead, Google DeepMind



Merve Noyan, Miquel Farré, Andrés Marafioti, and Orr Zohar
Foreword by Lucas Beyer

Vision Language Models

by Merve Noyan, Miquel Farré, Andrés Marafioti, and Orr Zohar

Copyright © 2026 Merve Noyan, Andrés Marafioti, Miquel Farré, and Orr Zohar. All rights reserved.

Published by O'Reilly Media, Inc., 141 Stony Circle, Suite 195, Santa Rosa, CA 95401.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://oreilly.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

Aquisitions Editor: Nicole Butterfield

Development Editor: Sara Hunter

Production Editor: Elizabeth Faerm

Copyeditor: nSight, Inc.

Proofreader: Piper Content Partners

Indexer: Judith McConville

Cover Designer: Susan Brown

Cover Illustrator: José Marzan Jr.

Interior Designer: David Futato

Interior Illustrator: Kate Dullea

June 2026: First Edition

Revision History for the First Edition

- 2026-06-08: First Release

See <http://oreilly.com/catalog/errata.csp?isbn=9798341624047> for release details.

The O'Reilly logo is a registered trademark of O'Reilly Media, Inc. *Vision Language Models*, the cover image, and related trade dress are trademarks of O'Reilly Media, Inc.

The views expressed in this work are those of the authors and do not represent the publisher's views. While the publisher and the authors have used good faith efforts to ensure that the information and instructions contained in this work are accurate, the publisher and the authors disclaim all responsibility for errors or omissions, including without limitation responsibility for damages resulting from the use of or reliance on this work. Use of the information and instructions contained in this work is at your own risk. If any code samples or other technology this work contains or describes is subject to open source licenses or the intellectual property rights of others, it is your responsibility to ensure that your use thereof complies with such licenses and/or rights.

979-8-341-62404-7

[LSI]

Foreword

When a research field is this young, it is tempting to describe it through its milestones. I've had the privilege to have a front-row seat for, and sometimes contribute to, each of this field's milestones:

- The realization that large-scale pretraining enables magic (BigTransfer)

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

- That the exact transformer architecture from language can work for vision (ViT)

I will never forget the night CLIP was published, marvelling at the possibilities training on images and free-form text opens up.

Finally, I've witnessed LLaVA, Flamingo, and PaLI be the sparks of the multimodal chatbots that we now have.

At the time, each of these steps felt simultaneously, absolutely unexpected and completely obvious in hindsight.

That is how progress feels when you are close to it.

But milestones are not the same as understanding. A paper can tell you what worked. A model card can tell you what was released. A demo can make the result feel effortless. None of these, by themselves, teaches you how to build a vision language model (VLM), how to debug it when it fails, or how to make the many small decisions that can make or break the final system.

That is where this book comes in. Merve, Miquel, Andi, and Orr do not treat any of this as magic. They open the field up. They show how VLMs are trained, how the data is prepared, what post-training means, and what deployment looks like. But not just that; they also explain how the same ideas and principles extend to documents, videos, any-to-any “omni” models, agents, all the way to robotics (VLAs).

A survey might have enumerated all the topics this book covers, and placed existing works into various buckets. This book, however, explains each topic well enough that the reader can understand the options and trade-offs to be made, and make their own choices. This is the book's spirit. It equips the reader with the necessary vocabulary to become an effective practitioner. The authors come from the open source multimodal community, and that shows: the explanations are practical without being shallow, the code is not an afterthought, and the goal is not to impress “reviewer number two” with how complicated the field is.

For me, the most exciting thing about VLMs is that they connect the concrete world of perception with the abstract world of language and thought. That connection brings machine learning closer to real-world impact. This book prepares readers for exactly that. It provides enough historical context to understand why VLMs look the way they do, enough engineering detail to work with them, and enough perspective to keep following the field as it moves.

I hope it helps many people go from being impressed by VLMs to being able to reason about them, improve them, and deploy them in useful systems.

Lucas Beyer

Coauthor of “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”

Preface

Today, you can take your phone out in a museum, snap a picture of a painting, and ask a model about the influences the artist drew on and what the piece might be trying to convey. The same model can watch the videos on your phone and give you quick summaries to help you find them later. Vision language models (VLMs) make all of this possible by connecting visual perception and language. They have moved quickly from research prototypes to real products that people use every day.

But building new things with these models is harder than the user experience suggests. The field moves fast, new articles come out daily, and practical guidance is scattered across blog posts, library docs, and informal knowledge passed around at networking events. If you want to use, train, or fine-tune a VLM, it is not obvious how to choose the right architecture, how to curate your datasets, or how to deploy efficiently. You end up piecing the knowledge together yourself.

This book is our attempt to change that. It is the book we wished we had when multimodal work stopped being a research curiosity and became an engineering problem.

We wrote it as a team that has spent years building, documenting, and shipping open source multimodal systems at Hugging Face. Between us we have trained and released VLMs like SmolVLM, integrated dozens of multimodal models into the open source ecosystem, built tooling and demos that make these models accessible to practitioners, and written extensively about the ideas behind them. That experience shaped the book: we lead with code and concrete examples, and we use theory to explain why things work (or don't) rather than to impress.

The book follows a deliberate arc. The first half takes you from foundations through training a VLM from scratch, real-world data curation, post-training, core architectures, and deployment at scale. The second half moves into specialized domains: document AI, video-language models, any-to-any systems, and agentic VLMs that move from passive understanding into decision making and action.

Who Is This Book For?

This book is for machine learning engineers, researchers, and technically minded builders who want to work with modern vision-language systems in practice. You may already use multimodal models through APIs or open-weight checkpoints but want to understand what is happening under the hood and build systems of your own.

It is not a complete introduction to machine learning from first principles. We assume you are comfortable with Python, notebooks, and some basic machine learning concepts. Most examples use PyTorch and the Hugging Face ecosystem, so prior exposure to those tools will help but is not mandatory. Familiarity with GPUs or cloud notebooks will make the hands-on parts easier.

Different chapters serve different goals. Some readers will care most about model training and post-training; others will come for deployment, document AI, video, or agents. The book is designed so that later chapters can stand on their own, but the early chapters provide the vocabulary and intuitions that make the rest of the journey much easier.

What You Will Learn

The goal of this book is to help you build, adapt, and reason about multimodal systems with confidence. We keep the discussion practical and use theory to illuminate design decisions rather than to overwhelm. By the end, you should be able to read a VLM paper critically, inspect a model card, and make informed decisions about architecture, data, post-training, and deployment for your own use case. More importantly, you should have a clear mental model of how these systems fit together, so that the next wave of papers and releases fits into a framework that you already understand:

Chapter 1, “Introduction to Vision and Language”

Sets the stage by tracing the ideas that led to modern VLMs and by showing how images and text came to be modeled together

Chapter 2, “Vision Language Model Applications”

Surveys the main tasks practitioners care about, including captioning, visual question answering, reasoning, retrieval, document understanding, video understanding, and localization-style applications

Chapter 3, “Vision Language Model Training”

Walks through training a small VLM from scratch so you can see what the model actually learns, how batching and packing work, and how images and text are represented during training

Chapter 4, “Training Data and Preprocessing for VLMs”

Moves from toy examples to real-world scale and shows how to source, filter, annotate, mix, and package multimodal datasets

Chapter 5, “Post-Training Vision Language Models”

Covers supervised fine-tuning, parameter-efficient adaptation, quantization-aware workflows, and alignment techniques that make models more helpful and better suited to specific tasks

Chapter 6, “Core Architectures of Vision Language Models”

Opens the model up and examines the main design patterns behind multimodal attention, fusion, and modern VLM blueprints

Chapter 7, “Deploying Models for Inference at Scale”

Focuses on the engineering side of inference: profiling, KV-cache behavior, attention optimizations, quantization, export, and serving frameworks

Chapter 8, “Document AI”

Shows how multimodal models handle OCR, document question answering, parsing, retrieval, and other document-centric workflows

Chapter 9, “Video-Language Models”

Extends the discussion to time by covering temporal modeling, video retrieval, Video-RAG, and practical fine-tuning considerations

Chapter 10, “Any-to-Any Models”

Explores unified multimodal systems that can understand and generate across text, images, audio, and video, along with the main architectural families behind them

Chapter 11, “Advanced Topics and Cutting-Edge Research”

Looks ahead to agentic VLMs and vision-language-action systems that move from passive understanding to decision making and action

Although the chapters are broadly self-contained, the book progresses from foundations to specialized systems. Readers who are new to the area will get the most out of the book by starting with the earlier chapters before jumping to the later application domains.

Conventions Used in This Book

The following typographical conventions are used in this book:

Italic

Indicates new terms, URLs, email addresses, filenames, and file extensions.

`Constant width`

Used for program listings, as well as within paragraphs to refer to program elements such as variable or function names, databases, data types, environment variables, statements, and keywords.

Constant width bold

Shows commands or other text that should be typed literally by the user.

Constant width italic

Shows text that should be replaced with user-supplied values or by values determined by context.

TIP

This element signifies a tip or suggestion.

NOTE

This element signifies a general note.

WARNING

This element indicates a warning or caution.

Using Code Examples

Supplemental material (code examples, exercises, etc.) is available for download at <https://huggingface.co/vlmbook>, including code examples and sample datasets.

If you have a technical question or a problem using the code examples, please send email to support@oreilly.com.

This book is here to help you get your job done. In general, if example code is offered with this book, you may use it in your programs and documentation. You do not need to contact us for permission unless you’re reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing examples from O’Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product’s documentation does require permission.

We appreciate, but generally do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: “*Vision Language Models* by Merve Noyan, Miquel Farré, Andrés Marafioti, and Orr Zohar (O’Reilly). Copyright 2026 Merve Noyan, Andrés Marafioti, Miguel Farré, and Orr Zohar, 979-8-341-62404-7.”

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

O'Reilly Online Learning

NOTE

For more than 40 years, **O'Reilly Media** has provided technology and business training, knowledge, and insight to help companies succeed.

Our unique network of experts and innovators share their knowledge and expertise through books, articles, and our online learning platform. O'Reilly's online learning platform gives you on-demand access to live training courses, in-depth learning paths, interactive coding environments, and a vast collection of text and video from O'Reilly and 200+ other publishers. For more information, visit <https://oreilly.com>.

How to Contact Us

Please address comments and questions concerning this book to the publisher:

O'Reilly Media, Inc.

141 Stony Circle, Suite 195

Santa Rosa, CA 95401

800-889-8969 (in the United States or Canada)

707-827-7019 (international or local)

707-829-0104 (fax)

support@oreilly.com

<https://oreilly.com/about/contact.html>

We have a web page for this book, where we list errata and any additional information. You can access this page at <https://oreil.ly/vision-language-models>.

For news and information about our books and courses, visit <https://oreilly.com>.

Find us on LinkedIn: <https://linkedin.com/company/oreilly>.

Watch us on YouTube: <https://youtube.com/oreillymedia>.

Acknowledgments

We are especially thankful to the lineup of reviewers that studied the book and helped us make it stronger: Lewis Tunstall, Igor Petrovski, Giuseppe Manzari, Ahmed Nassar, Dana Aubakirova, Xiaohan Wang, Quentin Gallouédec, and Pedro Cuenca. Thank you!

Merve

I believe teaching is one of the most impactful things one can do: it gives one the power to alter the course of someone's life. Any opportunity to teach is a gift that should be treated with utmost care.

During the early years of my career, I learned a lot from O'Reilly books, and it has been a dream to be able to give back, so I'm very grateful for this opportunity. Writing this book alongside the incredibly talented Miquel, Andi, and Orr was joyful, and I could not have asked for a better set of people to do this with!

I would like to thank my family, my mom Makbule and my grandma Sevgiser for their years of unwavering support, my dad Kubilay for having my back, and my best friend Doğa, who was always there for me whenever I needed support.

Lastly, I am very lucky to have crossed paths with Pedro, Lysandre, Julien, and Omar. I will be forever grateful for their continuous guidance and support throughout my journey in open source.

Miquel

Plant a tree, have a child, write a book...done. This book was such a great excuse to keep learning and spend more time with the fun and talented Andi, Merve, and Orr. What an experience!

It would have been impossible without my wife Viviane: juggling kids and writing on evenings and weekends is challenging, and while I tried to be there, sometimes I needed an extra hand and hers was always ready.

Last but not least, I have been lucky to be surrounded by people that I admire and that became friends. In the pinball of life, those who recently steered me: Leandro, Lewis, and Andi (a.k.a. the Hugging Face Bern core team), Cesar Allende (partner in crime on downhill biking). And those who keep steering me: Ferran Marqués and Anthony Accardo (my lifelong mentors).

Andi

Around the time we finished the proposal for this book, I was blessed with the birth of my first child, Elliot. I wrote large chunks of it wearing him in a baby carrier while my rock of a partner, Simone, caught up on some sleep. Writing a book with a newborn put real pressure on our young family, and I'm so grateful to her for always daring me to reach further and dream larger. Simone, I owe you the time, and I can't wait to return the favor when you write yours.

To my coauthors, Merve, Miquel, and Orr: each of you brought a unique and essential perspective to this book. I'm thankful you all agreed to join me in this wild journey, and I'm grateful for how much closer it brought us.

Orr

Writing this book with Merve, Miquel, and Andi was one of those rare collaborations where the work itself was the reward. Thank you all for being endlessly supportive along the way.

Roni, you were a sounding board, a source of patience, and the person who held everything together while I wrote. Thank you for all of it.

My father, Dov Zohar, dedicated his life to a single idea and changed an entire field in the process. He taught me what curiosity looks like when you take it seriously. I wish he were here to read this.

Chapter 1. Introduction to Vision and Language

We are social and visual animals. We evolved in groups, depending on one another for survival. Our brains are wired for communication, empathy, and connection. Even our emotional well-being is deeply tied to our relationships. To strengthen our bonds, we developed language, a tool to optimize communication, and writing systems that allowed us to reach beyond the limits of face-to-face interaction.

Yet, among all our senses, vision dominates our perception of the world. A huge part of our brain is dedicated to processing visual information. We rely on sight to understand our environment, navigate spaces, recognize others, and pick up subtle social cues like facial expressions and body language.

It's no surprise, then, that when we set out to build intelligent systems, we were captivated by two grand challenges: teaching them *language* to communicate with us, and *vision* to perceive the world as we do. At first, our systems were limited to handling either language or vision but not both together. Over time, however, a set of breakthroughs allowed us to merge these two rich modalities into unified architectures, giving rise to what we now call *vision language models* (VLMs).

In this chapter, we will briefly explore the history of machine learning and computer vision that led to this new paradigm, and set the stage for the exciting advances ahead.

Today's VLMs can do things that would have seemed like science fiction just a few years ago. Consider a few examples:

Describe and reason about images

You can hand a model a photograph and ask “What’s happening here?” and get a detailed, contextual answer. Models like OpenAI’s GPT series and Google’s Gemini can interpret charts, extract data from receipts, compare product designs, and answer nuanced visual questions.

Understand long videos

Today’s VLMs can process hour-long videos, summarizing them or answering questions about what’s happening in the video.

Read text in images across languages

Modern VLMs can understand documents natively and recognize text in almost all languages, including handwritten notes.

These capabilities are stunning, but they also raise a natural question: “How do these models actually work?” The truth is, with closed models, we are clueless. But with open models, we can reverse engineer! Since open models, too, are an accumulation of years of scientific work and ideas, we need to first learn the main building blocks and a small history on “how we got to here.” In this chapter, we will go through a bit of history to see how we process images and text so far, so no matter how the paradigm shifts, you will grasp new technologies quickly. On top of this, you will see how the image and text can be bound to each other.

LEARNING OBJECTIVE

In this chapter you will learn about the fundamental ideas of processing image and text, and connecting them together. You will learn theory as well as the tools you need to put the theory into practice.

Brief Introduction to Computer Vision

Before the deep learning era, computer vision relied heavily on handcrafted feature extraction techniques. Modern methods inherit from earlier techniques, so understanding them is important.

Let’s briefly take a look at signal decomposition methods.

Signal Decomposition Techniques

While the name might sound intimidating, the idea behind signal decomposition is straightforward. Instead of working directly with an image’s pixel values, these techniques re-express the image using mathematical building blocks such as sines, cosines, and wavelets, making certain properties much easier to see and manipulate. Think of it like switching from prose to sheet music: the content is the same, but the new representation makes specific patterns (rhythm, harmony) immediately visible.

Three transforms matter most for understanding the history of computer vision.

Evaluation Only. Created with Aspose.Words. Copyright 2003-2026 Aspose Pty Ltd.

Fourier transform decomposes an image into a sum of sine and cosine waves at different frequencies. Low-frequency waves capture broad, smooth variations (overall lighting, gradual shading), while high-frequency waves capture sharp transitions like edges and fine textures. By working in this frequency representation, engineers could selectively keep or remove certain frequencies—for example, suppressing high-frequency noise while preserving the important structure underneath.

Discrete cosine transform (DCT) is a close relative of the Fourier transform, which packs most of the image's information into just a few low-frequency coefficients. Natural images are dominated by smooth regions and gradual gradients, so the high-frequency coefficients that encode fine detail carry very little energy. DCT exploits this property and is the cornerstone of JPEG compression.

Where Fourier and DCT tell you *which* frequencies are present in the entire image, wavelets tell you which frequencies are present and *where*. This spatial awareness makes wavelets effective for tasks where you need to preserve sharp edges while compressing smooth regions, as in JPEG2000, or for denoising without blurring larger structures.

Let's see one of these transforms in action. **Figure 1-1** shows a sample image and its DCT representation. The DCT works on small, fixed-size patches of the image (typically 8×8 pixels), and for each patch, it computes a grid of coefficients. The two axes, u and v , represent horizontal and vertical frequency. Low values of both correspond to smooth, slowly varying shading; high values represent fine checkerboard-like textures. The second panel in **Figure 1-1** shows that most energy clusters in the upper-left corner—exactly the low-frequency coefficients. JPEG compression takes advantage of this by discarding the small, high-frequency values (third panel). The result is a much smaller file that looks nearly identical to the original because our eyes are far more sensitive to smooth, low-frequency structures than to fine-grained textures.

These transforms laid the groundwork for how we think about visual information: not as raw pixel grids but as structured signals with patterns we can isolate and manipulate. The next question engineers faced was: how do you detect specific spatial patterns such as edges, corners, and textures directly in the image itself?

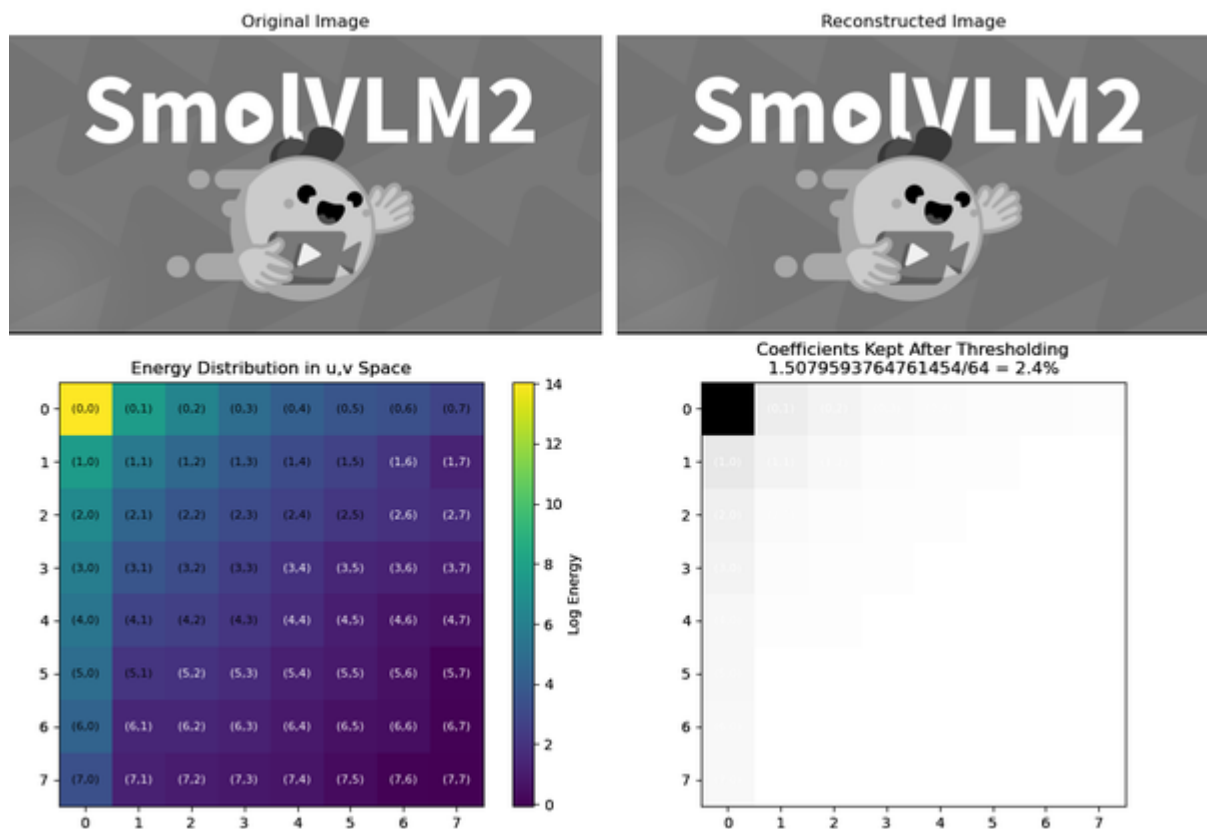


Figure 1-1. DCT compression pipeline.

Filters and Feature Extraction Kernels

The search for spatial patterns directly in images leads us to feature-extraction kernels and, eventually, to the two breakthroughs that define modern computer vision: convolutional neural networks (CNNs) and vision transformers (ViTs).

Before we get there, though, it is worth spending some time with kernels, because they are the conceptual bridge between classical signal processing and deep learning. The edge detectors and texture filters that engineers once designed by hand are exactly the kind of operations that CNNs learn to perform on their own during training. A CNN's earliest layers typically converge on filters that look remarkably similar to classical edge detectors. So understanding how handcrafted kernels work gives you direct intuition for what a CNN's first layers are actually doing and why architectures like ViTs later moved beyond them.

A feature extraction kernel is a small matrix (also called a filter) that is applied over an image to detect specific patterns. At each position, the kernel values are multiplied by the underlying pixel values, and the results are summed to produce a single new value. Slide the kernel one step at a time, repeat, and you build up an entirely new representation of the original signal: one that emphasizes whatever pattern the kernel was designed to find.

That sounds abstract, so let's make it concrete with the simplest possible example. Instead of a full two-dimensional image, consider a one-dimensional signal: a sequence of values that rises, holds steady, and then drops, as shown in [Figure 1-2](#).

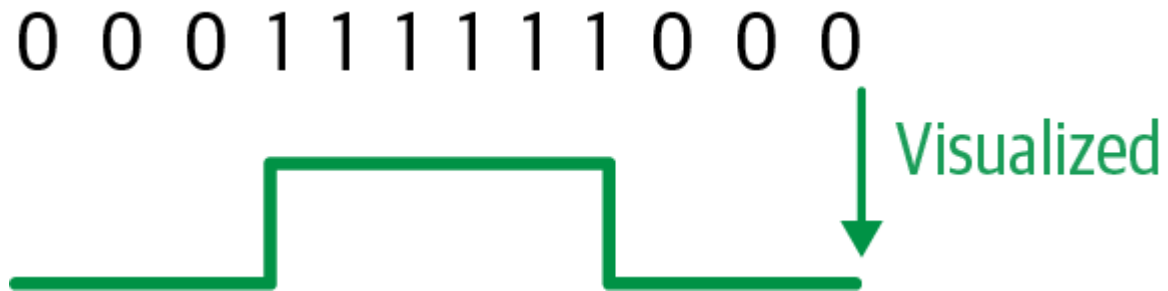


Figure 1-2. Visualization of a 1D signal.

Our goal in this example is to detect where this signal changes. To do that, we need a kernel that responds to differences between neighboring values. The simplest kernel that does this is $[-1, 1]$ just two numbers.

Let's see the kernel in action:

- When this kernel sits over a flat region (say, two consecutive 5s), it computes $(-1 \times 5) + (1 \times 5) = 0$: no change detected.
- When it faces a jump in the signal (say, from 3 to 7), it computes $(-1 \times 3) + (1 \times 7) = 4$: a positive spike that flags the transition.

This tiny kernel acts as a change detector. To apply it, we slide the kernel across the signal from left to right, computing one output value at each position, as illustrated in [Figure 1-3](#). At each step, we align the kernel with two adjacent signal values, multiply them element-wise, and sum the results. The resulting sequence of outputs is our filtered signal.

One detail to notice in [Figure 1-3](#): to produce an output at the very first position, we need a signal value to the left of our sequence where none exists. To handle this, we introduce a zero, a placeholder that lets us compute the output without losing the first position. This is called padding, a standard technique in convolution. Zero-padding (filling with zeros wherever the kernel extends beyond the signal boundary) is the most common strategy, though others exist.

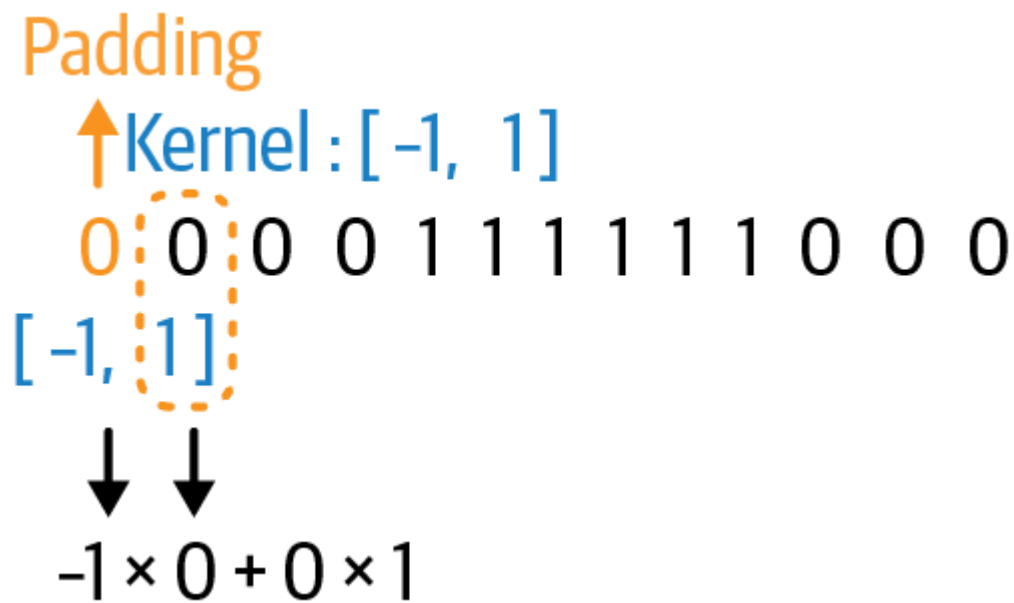


Figure 1-3. Applying a convolution to a sequence.

This slide-multiply-sum operation is called a convolution, and its output is called a feature map. That map highlights the specific feature the kernel was designed to find. In our case, the feature map spikes wherever the original signal transitions between values, and sits at zero or near zero wherever the signal is flat (see Figure 1-4).



Figure 1-4. Visual input and output of the convolution.

Figures 1-4 and 1-5 overlay the original signal and its feature map so you can see this relationship directly.

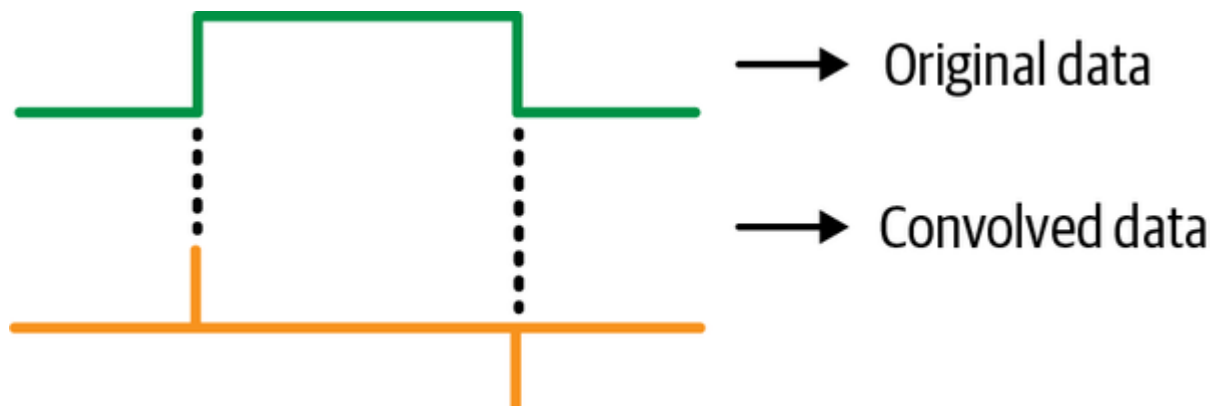


Figure 1-5. Visual input/output.

Now let's extend this idea from one dimension to two. A grayscale image is essentially a 2D signal (a grid of pixel intensities), so instead of a 1D kernel like $[-1, 1]$, we use a small 2D matrix, typically 3×3 . The operation is the same: slide the kernel across the image, multiply element-wise at each position, and sum to produce one value in the output feature map.

The choice of values in the kernel determines which patterns are detected. Figure 1-6 shows two classic examples: the Prewitt and Sobel kernels. Both are designed to detect edges: the Prewitt kernel in its simplest form, and the Sobel kernel with slightly more weight on the center row or column, which makes it more robust to noise. Each comes in horizontal and vertical variants, so together they can capture edges in any orientation. These kernels were workhorses of image processing for decades and remain useful reference points.

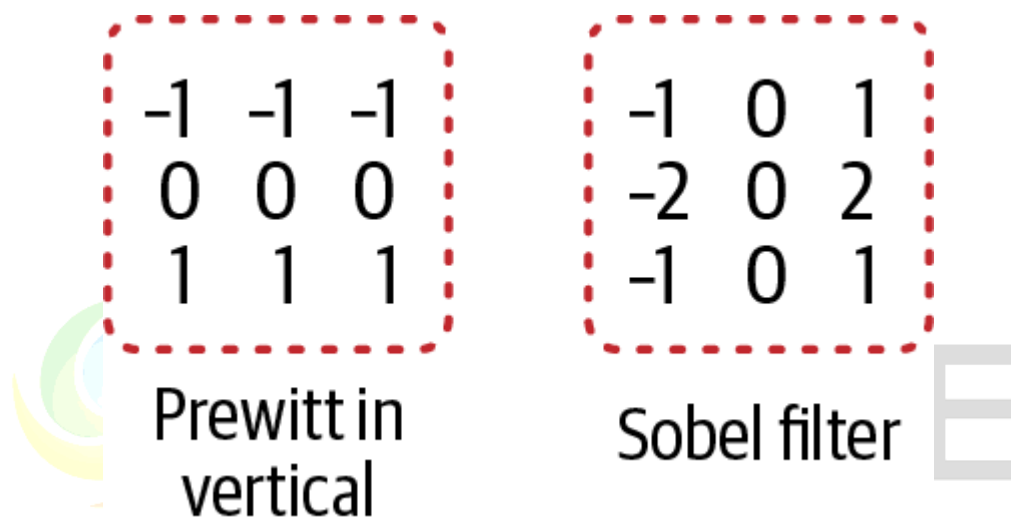


Figure 1-6. Prewitt and Sobel kernels for edge detection.

The key takeaway is that convolution gives us a principled way to extract meaningful structure (edges, corners, textures) from raw pixel data. The pattern you detect depends entirely on what values you put in the kernel. For the approaches we have seen so far, those values were chosen by engineers. The question that drove the next era of computer vision was this: what if the kernel values could be learned automatically from data?

This document was truncated here because it was created in the Evaluation Mode.